

仓颉代码片段语义检查-设计文档

赛题名称：仓颉代码片段语义检查

竞赛：2026 年全国大学生计算机系统能力大赛——编译系统挑战赛

队伍：圆周运动（T2026100552010674）

学校：南开大学

目录

- 仓颉代码片段语义检查-设计文档
 - 目录
 - 评审速览：作品特色与直接证据
 - 摘要
 - 1. 问题分析与设计目标
 - 1.1 逐 token 前缀判断
 - 1.2 设计目标与范围
 - 1.3 官方材料协议差异与兼容
 - 2. 五阶段版本演进
 - 3. 当前加固纯 C++ 架构与功能
 - 3.1 构建期与运行时数据流
 - 3.2 主要模块
 - 3.3 功能覆盖
 - 4. 核心技术贡献与加固版伪代码
 - 4.1 稳定词法事实与可延长词素分流
 - 4.2 前缀可续写性判定模型
 - 4.2.1 定义与状态
 - 4.2.2 不可恢复错误判定
 - 4.2.3 PartialLexeme 处理原则
 - 4.2.4 设计不变量
 - 4.2.5 复杂度与最坏情况
 - 4.3 活动语句缓存覆盖多行表达式
 - 4.4 传递泛型接口要求与 alpha-renaming
 - 5. 重难点、修改历程与解决方案

- 5.1 BPE 分片与仓颉词法边界
- 5.2 合法未完成前缀的提前误报风险
- 5.3 每 token 重复扫描与深度解析
- 5.4 多行函数签名进入语义模型
- 5.5 嵌套 lambda 参数可见性
- 5.6 泛型接口传递继承与签名规范化
- 5.7 重载方法引用歧义
- 5.8 函数作用域与关键字边界
- 5.9 隐藏样例覆盖评估
- 6. 性能测试与 XGrammar 基线对比
 - 6.1 实验环境与统计方法
 - 6.2 五阶段平台性能
 - 6.3 内部原生化版本的冷进程辅助数据
 - 6.4 与类似方法和参考工作的对比
- 7. 正确性与鲁棒性测试
 - 7.1 验证方法
 - 7.2 分层验收结果
 - 7.3 代表性复现命令
- 8. 为什么选择加固纯 C++
 - 8.1 团队的工程取舍
 - 8.2 加固重点降低的潜在风险
- 9. 已知边界与适用范围
- 10. 第三方来源与原创贡献声明
 - 10.1 可审计组件清单
 - 10.2 第三方能力、本队适配与独立实现边界
 - 10.3 竞赛配套 typechecker 的开发期用途与提交边界
 - 10.4 最终提交框架清单
- 11. 参考文献
- 12. 结论
- 附录 A: 逐例平台性能数据
 - 附录汇总

评审速览：作品特色与直接证据

从“可判断”到“可在线实时判断”

XGrammar 通用语法约束与本队原生 C++ 前缀语义规则协同工作，重点控制合法前缀的提前误报。

50 / 50

公开错误样例

精确命中首个不可恢复 token

360 / 1800 / 0

随机程序与分片

程序数 / 分片执行数 / 失败数

加固纯 C++

当前竞赛热路径

增量词法、类型、作用域、调用与泛型

8.55x / 近10x

平台平均提速

相对 XGrammar + 仓颉简单适配基线

核心技术贡献：在不完整输入中分流稳定词法事实与可延长词素，并结合缓存模型执行选择性语义探测

任意 BPE / UTF-8 分片

稳定词法 TokenEvent

PartialLexeme 保留

缓存模型选择性 Probe

本作品在 XGrammar 通用语法约束能力和本队完成的仓颉语言简单适配基线上，构建了原生 C++ 增量前缀语义检查路径。系统使用 TokenEvent 固化已经确定的词法边界，使用 PartialLexeme 保存仍可延长的末尾内容。语义层结合完整可见源码、缓存的声明模型、函数上下文和活动语句执行选择性探测；普通输入轮次复用已有状态，类、接口、函数或语句达到提交点时更新相关模型。

评审可优先关注以下实验结果：公开错误样例精确识别 50/50；四个独立随机种子生成 360 个程序并完成 1800 次分片执行，失败数为 0；其中 180 个合法程序在全部观测前缀上实现零误报；最终版本在平台提交测试，平均耗时为 0.27328s，相对 XGrammar + 仓颉简单适配基线的 2.33746s 提速 8.55x，接近一个数量级。

摘要

赛题要求检查器逐个接收 cl100k_base token ID，并在每轮输入后判断当前仓颉源码前缀能否通过后续输入补全为合法程序。赛题说明文字使用 1=可续写、0=不可续写，组委会公开 checker 的实际交互协议使用 0=继续、1=首错。无参数运行的 solution 遵循公开 checker 的实际协议，并已据此取得平台 AC；--competition-output 兼容模式将输出翻转为赛题说明文字中的约定。两种模式共用相同检查状态和首错位置。

项目采用“XGrammar 字符级语法约束 + 原生 C++ 前缀语义检查”的双层架构。XGrammar 负责 GBNF 语法状态推进；本队实现仓颉 GBNF、cl100k token 解码适配、增量词法、稳定 token 与未完成词素分流、类型/作用域/调用/泛型/lambda/接口规则、构建期上下文二进制化，以及差分和随机化验证工具。当前竞赛运行路径由 C++ 进程、XGrammar 和 TVM FFI 共享库组成。

同一批 50 个公开案例中，XGrammar + 仓颉简单适配基线累计耗时 116.873s、平均 2.33746s；最终版累计耗时 13.664s、平均 0.27328s，平均耗时降低 88.31%。全部 50 个案例均取得正向加速，逐例提速范围为 3.87x–12.63x。

1. 问题分析与设计目标

1.1 逐 token 前缀判断

本题采用逐 token 的前缀可续写性判断。BPE token 边界与仓颉词法 token 边界相互独立，一个标识符、数值、多字符运算符、UTF-8 字符、字符串或嵌套注释可能跨越多轮输入。检查器需要在语义事实充分时尽早锁存错误，同时为仍有合法延长方式的片段保留补全空间。

例如，输入到标识符中途时，该串可能在下一轮继续构成可见名称；输入到字符串、函数参数列表或 lambda 表达式中途时，程序结构仍在形成。已经完成的 let 变量重新赋值、循环外 break、确定无法匹配候选函数的调用等情形，则可在当前轮形成不可恢复错误。

1.2 设计目标与范围

- 每轮输入后立即输出可续写性判断，并在第一次不可恢复错误处锁存失败状态；
- 覆盖赛题涉及的增量词法、语法、声明、作用域、类型、控制流、调用、泛型、类/接口和 lambda 子集；
- 精确报告公开错误样例的首错 token，并验证多种输入分片方式的一致性；
- 将竞赛热路径收敛到原生 C++ 进程，减少解释器启动、完整前缀扫描和深度解析；
- 以合法程序的全部观测前缀零误报作为与精确首错同等重要的验收标准；
- 保持第三方来源、团队修改和团队独立实现的归属清晰、可复查。

本作品面向赛题规定的仓颉语言子集。文中结论限定在公开语料、项目语料、随机生成语料、分片测试、sanitizer 测试和已记录的平台提交结果范围内。

1.3 官方材料协议差异与兼容

组委会公开的 [cangjie-fragment-checker](#) 使用逐 token stdin/stdout 交互，其 README 和 scripts/token_interaction_test.py 明确采用 0=当前无错、1=在本轮报告首错。公开 checker 的典型调用方式为：

```
python3 scripts/token_interaction_test.py wrong/err_arity.cj --cmd ./solution
```

公开 checker 不附加输出翻转参数，因此本项目默认启动命令和输出如下：

场景	启动方式	可续写	不可续写
公开 checker 与当前平台 AC 路径	./solution	0	1
赛题说明文字约定	./solution --competition-output	1	0

--competition-output 只在最终输出处翻转布尔编码，不改变 token 解码、语法状态、语义状态和首错位置。最终提交默认模式以平台实际 checker 协议为准；若组委会后续统一为赛题文字约定，可直接调整启动参数，无需重新实现检查算法。

2. 五阶段版本演进

下表中的五个阶段均包含本队完成的语言适配或优化工作。XGrammar 通用语法引擎归属于第三方项目；混合稳定版、未加固纯 C++ 版和当前加固纯 C++ 版构成本队连续完成的三个原生迭代。

阶段	版本	平台平均 耗时	相对 基线	平台数据对应	本队实现内容
1	XGrammar + 简单适配基线	2.33746s	1.00×	公开 50 例逐例记录	完成仓颌 GBNF、token 适配和初始语义检查；每个输入 token 会触发完整前缀扫描、Lark 解析和类型检查。
2	纯 Python 优化版	0.65642s	3.56×	2026-08-04 20:33:48, AC、25.00 分	默认热路径停用每 token Lark 深检；缓存函数、类、接口和上下文。
3	混合稳定版	0.31690s	7.38×	公开 50 例平台平均记录	C++ 负责竞赛协议、cl100k 解码和 XGrammar 字符级语法状态；Python worker 负责增量语义；后续优化进程通信和数据传输。
4	未加固纯 C++ 版	0.26866s	8.70×	公开 50 例平台平均记录	将增量词法、类型、作用域、调用、泛型和 lambda 等语义热路径迁移到 C++，竞赛运行时移除 Python worker。
5	当前加固纯 C++ 版	0.27328s	8.55×	2026-08-05 02:59:28, AC、25.00 分	增加多行声明、嵌套 lambda、泛型接口继承、方法泛型签名、重载方法引用和作用域隔离等规则。

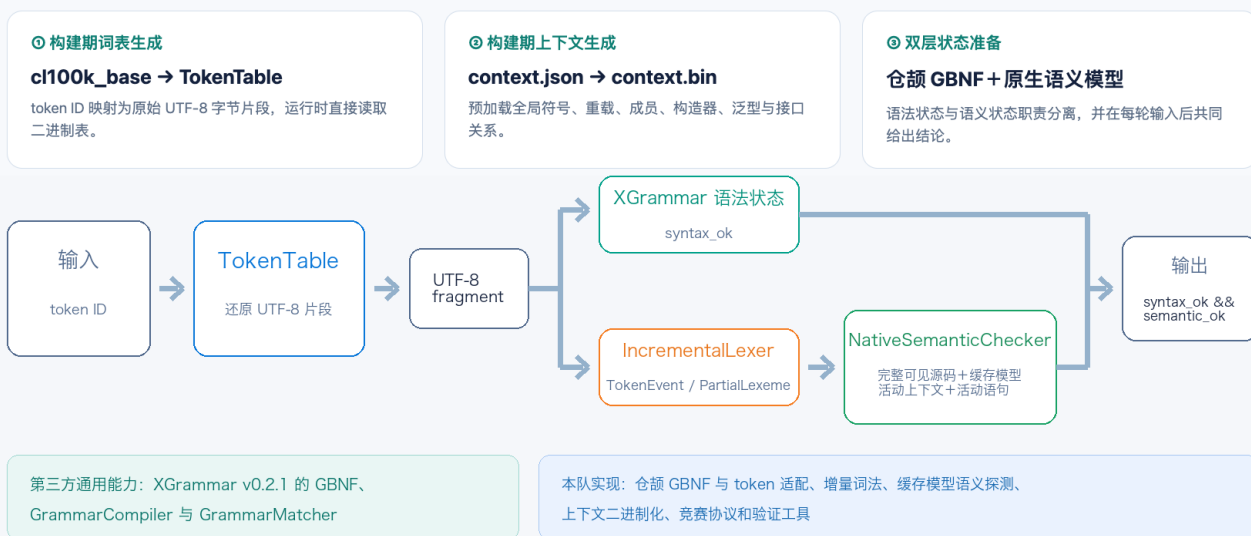
初始基线到纯 Python 优化版获得 3.56× 加速；纯 Python 到混合稳定版再获得 2.07× 加速；混合稳定版到未加固纯 C++ 版获得 1.18× 加速。当前加固版的平均耗时相对未加固版增加 1.72%，对应多行、lambda、接口、重载和作用域等隐藏样例覆盖能力的增加。最终版本相比基线，实现了约 8.55× 的加速。

3. 当前加固纯 C++ 架构与功能

3.1 构建期与运行时数据流

当前加固纯 C++ 版：双层增量检查架构

构建期生成紧凑二进制数据；运行时由 XGrammar 管理字符语法状态，本队 C++ 引擎管理仓颌语义状态。



构建期将竞赛 context 和 cl100k_base 词表预处理为二进制表：tools/generate_context_table.py 生成 generated/context.bin，tools/generate_cl100k_table.py 生成 generated/cl100k_base.bin。运行时读取二进制数据和 GBNF 语法，token ID 经 TokenTable 还原为 UTF-8 字节片段，随后进入 XGrammar 字符级语法状态和本队 C++ 语义路径。

```

token ID
  → C++ TokenTable
  → UTF-8 bytes
    ├── NativeSyntaxChecker / XGrammar GrammarMatcher
    │   └── syntax_ok
    └── IncrementalLexer
        ├── stable TokenEvent
        └── PartialLexeme
            ↓
            NativeSemanticChecker
            ├── complete visible source
            ├── cached Model / FunctionContext
            ├── ExpressionTyper
            └── ActiveStatementCache
                ↓
                semantic_ok
  → syntax_ok && semantic_ok
  → stdout
  
```

3.2 主要模块

模块	职责	实现要点
TokenTable	token ID 到 UTF-8 字节片段的还原	构建期生成二进制表，运行时直接查表
XGrammar 语法层	推进仓颉 GBNF 字符级语法状态	使用通用 GrammarCompiler / GrammarMatcher 能力
IncrementalLexer	识别稳定 token 与可延长片段	支持任意字节分片、UTF-8、注释、字符串、数值和多字符运算符
NativeSemanticChecker	保存稳定词法事件并执行 partial/source probe	首错锁存，复用源码、缓存模型、活动上下文和活动语句
Model / FunctionContext	保存预定义符号、声明、作用域和类型关系	context 构建期二进制化，结构提交点更新活动模型
ExpressionTyper	推导表达式、调用、泛型和 lambda 类型	支持期望类型传播、重载候选与类型变量替换
ActiveStatementCache	维护当前活动语句	增量跟踪括号、字符串、注释和多行结构
差分与生成工具	验证首错、分片、随机程序和性能	仅用于离线测试与复现

3.3 功能覆盖

下表中的“支持”限定在赛题仓颉子集、预置 context 和本文测试范围内，不外推到完整仓颉 1.0.0 语言。

赛题能力	支持范围	主要实现	测试证据
基础表达式与运算符	支持赛题整数、浮点、Bool、String、Rune、数组、索引、成员和常见运算符	ExpressionTyper	公开错误样例、单元测试、项目语料
if / while 条件	检查条件是否可兼容 Bool	FunctionContext、ExpressionTyper	err_if_not_bool、err_while_not_bool
for、range 与循环控制	检查 Array/ArrayList/HashMap/Range 等赛题可迭代类型、绑定模式和循环深度	循环上下文、可迭代类型检查	定向样例与随机控制流家族
变量与作用域	支持函数参数、函数体局部变量、let / var 可变性、已覆盖的重复声明与未定义符号、函数间隔离和活动 lambda 参数；结论不外推到完整仓颉语言的全部嵌套块遮蔽与生命周期组合	FunctionContext、活动 lambda 上下文和源码结构探测	公开样例、scope-isolation 随机家族、函数隔离回归
函数、方法与构造器调用	支持位置参数、命名参数、默认参数、静态/实例方法和赛题重载规则	Model、调用候选过滤	公开调用样例、overload 随机家族
class/interface	支持赛题范围内字段、方法、构造器、实现检查和传递泛型接口继承	nominal model、接口要求收集	接口公开样例、generic-inheritance 家族
显式泛型实参	支持数量检查、类型变量替换和具体化签名匹配	substitution、候选匹配	第一、二档公开样例与随机测试
泛型实参推导	根据实参、期望返回类型和候选约束推导赛题常见形式；歧义时拒绝唯一化	ExpressionTyper、类型变量统一	第三档公开样例、高阶泛型随机测试
显式参数类型 lambda	支持参数数量、参数/返回类型和高阶函数上下文	活动 lambda 上下文、期望函数类型传播	第二档 lambda 样例与固定回归
lambda 参数类型推导	从目标函数类型、重载候选和回调接口传播期望类型；无唯一候选时保留或报告歧义	候选函数类型、活动 lambda 参数	第三档 lambda 样例、nested-lambda 家族

本文的作用域结论限定在公开样例、平台测试及项目定向测试覆盖的函数参数、函数体局部变量、循环绑定、活动 lambda 参数和函数间隔离场景。当前设计文档不声称覆盖完整仓颉语言中的全部嵌套块遮蔽、复杂闭包捕获及非常规生命周期组合。

4. 核心技术贡献与加固版伪代码

GBNF、函数与循环上下文、alpha-renaming 和增量缓存属于通用编译技术。本队的技术贡献集中在这些技术面向赛题前缀判定的组合与落地：在 BPE token、UTF-8 字节和仓颉词法 token 三种边界之间建立稳定词法事件/未完成词素分流；将完整程序语法规则改造成保守的不可恢复错误判定；让 XGrammar 语法状态与原生 C++ 语义探测协同推进；在保留公开首错精度的同时消除每 token 全量解析；将 context 和 cl100k 词表移到构建期二进制化。

当前版本中的 TokenEvent 主要承担词法稳定性划分。语义层的增量性来自缓存模型、活动上下文、活动语句和按提交点触发的选择性重建，所有语义事实尚未转换为由 TokenEvent 执行的单次提交。

4.1 稳定词法事实与可延长词素分流

BPE token、UTF-8 字节和仓颉词法 token 具有不同边界。加固版使用 TokenEvent 表示已经稳定的词法边界，使用 PartialLexeme 保存仍可延长的末尾片段。语义层结合完整可见源码、缓存模型、活动上下文和活动语句执行选择性探测。

算法 1：增量前缀检查主流程

输入：本轮收到的 UTF-8 字节片段 fragment

状态：source、IncrementalLexer、SemanticEngine、failed

```
CHECK_FRAGMENT(fragment):
    if failed:
        return ERROR

    source ← source + fragment
    (stable_events, partial) ← lexer.FEED(fragment)

    for event in stable_events:
        semantic_engine.ACCEPT(event)

    status ← semantic_engine.PROBE(partial, source)
    if status is ERROR:
        failed ← true

    return status
```

PROBE 根据本轮新增字符决定活动模型和上下文的更新范围：

```
PROBE(partial, source):
    delta ← source 中本轮新增的部分

    if delta 包含类、接口、函数或结构提交标记：
        active_model ← preload_model
        收集新增 import、函数、类和接口

    if delta 包含作用域或语句边界：
        更新当前函数、局部变量、循环和 lambda 上下文

    active_statement ← statement_cache.UPDATE(source)
    return ANALYZE_SOURCE(
        source,
        active_model,
        model_dirty,
        commit_dirty,
        active_context,
        active_statement
    )
```

对应实现为 IncrementalLexer::Feed、NativeSemanticChecker::Check、IncrementalSemanticEngine::Accept、IncrementalSemanticEngine::Probe 和 ActiveStatementCache::Update。Accept 当前保存稳定词法事件，主要语义判断由 Probe 完成。Probe 使用 delta 判断模型和上下文是否需要更新，并对当前活动语句执行类型、调用和作用域检查。因此，本文所称“增量语义”指停用每轮 Lark/完整 typechecker 深检、复用声明模型与活动上下文，并按提交点选择性更新；当前实现尚未把全部语义规则转换成纯事件驱动状态机。

4.2 前缀可续写性判定模型

4.2.1 定义与状态

从赛题定义看，给定当前源码前缀 p ，若存在某个后缀 s ，使 $p + s$ 成为赛题仓颉子集中语法、类型和作用域均合法的完整程序，则称 p 可续写：

$$\text{CONTINUABLE}(p) \Leftrightarrow \exists s, \text{VALID_PROGRAM}(p + s)$$

当前实现不构造实际补全后缀，也不对完整仓颉语言给出形式化可达性证明。系统采用面向赛题子集的保守错误判定方法：未完成结构优先延迟判断；仅在语法状态无后继，或某条已实现语义规则能够确认后续输入无法修复时锁存错误。持久状态和单轮工作集可概括为：

```
PersistentPrefixState =  
    LexerState  
    × GrammarState  
    × GlobalModel  
    × FunctionContext  
    × ActiveLoopContext  
    × ActiveLambdaContext  
    × ActiveStatement  
    × FailedLatch  
  
ProbeWorkingSet =  
    PartialLexeme  
    × CandidateTypes  
    × CandidateCalls  
    × ExpectedLambdaType
```

CandidateTypes、CandidateCalls 和 lambda 期望类型在当前活动表达式的 probe 中构造；结构提交点到达后，相关声明模型和活动上下文按需更新。概念上，单轮判定分为三类：

内部结果	含义	对外判断
NO_ERROR_DETECTED	当前稳定事实没有触发已实现的不可恢复错误规则	可继续
DEFERRED_INCOMPLETE	当前词素、表达式、调用或 lambda 尚未闭合，语义判断被延迟	可继续
IRRECOVERABLE_ERROR	已满足某条后续输入无法修复的错误条件	错误并锁存

当前 C++ 接口使用 CheckStatus.ok 对外返回，因此 NO_ERROR_DETECTED 和 DEFERRED_INCOMPLETE 均对应 ok=true，IRRECOVERABLE_ERROR 对应 ok=false。返回“可继续”表示当前没有触发已实现的不可恢复错误规则，或结构尚未闭合而需要延迟判断。公开 50 例精确首错、平台 AC、合法前缀零误报和定向随机测试用于验证该工程判定在赛题负载下的有效范围。

4.2.2 不可恢复错误判定

语义检查仅在某条已实现规则确认当前错误不受未来后缀影响时锁存错误；无法确定时采用延迟判断。主要判定条件如下：

错误类别	最早判定条件	后续输入无法修复的原因
语法自动机无后继	XGrammar 拒绝本轮新增字节，当前语法状态不存在对应转移	已输入字符无法删除或改写
break / continue 越界	关键字已经稳定，当前函数的循环深度为 0	后续新增循环无法包围已经出现的控制语句
let 重新赋值	左值和赋值语句达到提交点，符号已确定为不可变	后缀无法改变已有声明的可变性
未定义标识符	标识符词素与所在表达式已稳定，且不属于可见符号、类型、关键字或其可延长前缀	后缀无法修改已经提交的名称文本和既有可见域
条件类型错误	if / while 条件闭合，推导类型与 Bool 不兼容	后缀无法重新打开已闭合条件表达式
调用无候选	参数列表闭合，全部重载因名称、数量、命名参数或类型不匹配被排除	后缀无法向已闭合调用补入参数或恢复候选
数组元素类型错误	目标元素类型已知，当前元素的确定类型与目标类型不相交，并已达到相应提交条件	后缀只能继续外层结构，无法改变已确定元素的类型类别
返回类型错误	return 表达式达到语义提交点，推导类型与函数结果类型不兼容	后续语句无法修改已提交的返回表达式
重复声明	声明名称、声明种类和所在作用域已经确定	后缀无法撤销同一作用域中的既有声明

以 `let a: Array<Int64> = [` 为例，首个引号已经提供 `String` 类型类别信息，字符串内容仍处于未完成状态。实现会结合目标类型、数组活动语句和语义提交条件保留或消除候选；当 `String` 元素类别已经固定、目标 `Int64` 已确定且无赛题允许的转换路径时，类型冲突成为不可恢复错误。不同 `cl100k` / 字节分片可能改变该条件在第几轮凑齐，公开首错 token 标注用于约束平台协议下的最终拒绝轮次。默认公开 checker 协议在拒绝轮输出 1，`--competition-output` 模式在同一轮输出 0。

4.2.3 PartialLexeme 处理原则

- 未完成标识符：当文本仍是可见符号、类型名或关键字的前缀时延迟判断；分隔符到达后再提交完整名称。
- 未完成数值：保留整数、浮点数、后缀和指数形式等仍可形成的候选；数值词素稳定后提交具体类型。
- 未完成字符串：保留字符串闭合和转义状态，同时允许表达式层使用已经确定的 `String` 类型类别做保守推导。
- 未完成运算符：`<`、`>`、`=`、`.` 等可能继续形成多字符运算符，在最长可行形式稳定前保留语法分支。
- UTF-8 中间字节：完整码点形成前只推进字节 / 词法状态，不提交语义名称或字面量。
- 未闭合调用或 `lambda`：保留仍可满足的重载、参数类型和期望返回类型；闭合前避免任意选定单一候选。

4.2.4 设计不变量

1. 已提交的 `TokenEvent` 不会被未来输入改变词素边界或类别；
2. 未闭合且当前无法确定错误不可恢复的结构采用延迟判断；
3. 仅在某条已实现规则确认当前错误不受未来后缀影响时锁存错误；

- 4. 错误锁存后，后续输入不会恢复为可续写状态；
- 5. 分片布局只影响事件到达轮次，拼接后的相同字符流应得到一致的最终结论。

4.2.5 复杂度与最坏情况

令 Δ 为本轮新增字节数， n 为当前累计源码长度， a 为活动语句长度， c 为调用/重载候选数。

环节	常见开销	最坏情况与触发条件
token ID 解码	查表后复制本轮字节，约 $O(\Delta)$	与解码片段长度线性相关
XGrammar 状态推进	与新增字节数及语法状态转移代价相关	复杂语法状态下转移代价增加
<code>IncrementalLexer::Feed</code>	扫描新增字节和当前 pending 尾部，约 $O(\Delta + \text{pending})$	超长未闭合字符串/注释会重复扫描增长中的 pending，累计可能达到二次量级
<code>ActiveStatementCache::Update</code>	函数体稳定时从 cursor 扫描新增字节，摊销接近 $O(\Delta)$	函数上下文切换或缓存重置后需要重新建立活动状态
表达式与调用分析	约 $O(a \times c)$ ，具体取决于嵌套表达式和候选过滤	超长活动表达式、大量重载或嵌套泛型会放大开销
模型与上下文更新	大多数普通字节复用旧模型	类、接口、函数或结构提交点可能触发对当前源码的局部模型重建或较大范围扫描，单轮可达 $O(n)$
当前 Probe 边界扫描	每轮仍需计算部分结构状态	当前实现中部分花括号/上下文探测会扫描可见源码，单轮最坏为 $O(n)$ ；后续可用持久化结构帧继续优化

因此当前实现的平均路径具有明显增量收益，最坏路径仍保留全前缀或增长 pending 的重复工作。平台 8.55× 数据反映赛题公开负载下的实际综合结果，未将所有操作概括为常数复杂度。

4.3 活动语句缓存覆盖多行表达式

`ActiveStatementCache` 从上次 cursor 继续扫描本轮新增字符，维护圆括号、方括号、花括号、字符串、转义、行注释和嵌套块注释状态。语句提交仅发生在顶层分号或满足条件的顶层换行位置。

算法 2：活动语句增量维护

`UPDATE(source):`
 从上次 cursor 继续扫描新增字符

 遇到字符串或注释：
 更新字符串、转义和注释状态

```

遇到左括号、方括号或花括号：
    对应深度加一

遇到匹配的右括号：
    对应深度减一

if 当前位于顶层 and 遇到分号或可提交换行：
    提交上一条完整语句
    更新 statement_start

return source[statement_start : 当前可见末尾]

```

该缓存覆盖多行变量声明、多行调用、嵌套 lambda，以及字符串和注释中的括号。语义层由此获得结构完整的活动语句视图。

4.4 传递泛型接口要求与 *alpha-renaming*

加固版沿接口继承链替换类型实参，使用 visited 集合控制重复与循环访问；方法签名比较前，将接口方法的泛型参数映射到实现方法的泛型参数，实现 *alpha-renaming*。

算法 3：泛型接口实现检查

```

COLLECT_REQUIREMENTS(interface_type, visited):
    interface_type ← 规范化(interface_type)

    if interface_type in visited:
        return
    visited.add(interface_type)

    interface ← 查找接口定义
    substitution ← 接口类型参数到实际类型实参的映射

    for required_method in interface.methods:
        requirements.add(
            SUBSTITUTE_SIGNATURE(required_method, substitution)
        )

    for parent_interface in interface.supers:
        concrete_parent ← SUBSTITUTE(parent_interface, substitution)
        COLLECT_REQUIREMENTS(concrete_parent, visited)

SAME_SIGNATURE(implementation, requirement):
    if 泛型参数数量或方法参数数量不同:
        return false

    rename ← requirement 方法泛型参数
        到 implementation 泛型参数的逐项映射
    normalized_requirement ← SUBSTITUTE_SIGNATURE(requirement, rename)

```

```
return 参数类型逐项相同
and 返回类型相同
```

对应实现为 `CollectInterfaceRequirements`、`SubstituteSignature` 和 `SameInterfaceSignature`。该路径支持 `Interface<T>` 的传递继承、具体化接口要求和方法泛型参数规范化。

5. 重难点、修改历程与解决方案

5.1 BPE 分片与仓颉词法边界

- 现象：标识符、UTF-8 字符、数字、多字符运算符、字符串和嵌套注释可能跨越多轮输入。
- 根因：LLM token 边界无法直接作为仓颉 lexer 的提交边界。
- 解决方法：C++ `IncrementalLexer` 维护 pending buffer，输出稳定 `TokenEvent`，将末尾可延长内容保留为 `PartialLexeme`。
- 验证证据：逐字节、固定种子随机、cl100k 和整段四种分片得到一致结果。

5.2 合法未完成前缀的提前误报风险

- 现象：标识符、字符串、函数参数或 lambda 的中间状态仍有合法补全路径。
- 根因：完整程序 typechecker 的诊断对象是完整语法结构，前缀检查还需判断未来延长空间。
- 解决方法：稳定词法边界保存、partial probe、可见符号前缀判断和首次错误锁存协同工作。
- 验证证据：180 个合法程序在全部观测前缀上零误报；变异样例在 `mutation_start` 前保持与合法基线一致的判断。

5.3 每 token 重复扫描与深度解析

- 现象：初始基线的 50 例平均耗时为 2.33746s。
- 根因：每轮输入都会重新扫描完整源码前缀，并执行 Lark 解析和类型检查。历史性能分析记录了公开案例约 5575 次深检，Lark/typechecker 约占原热路径 83%。
- 解决历程：纯 Python 版缓存声明与上下文；混合版迁移协议、解码和 XGrammar 状态；纯 C++ 版迁移语义热路径；构建期生成 cl100k 和 context 二进制表。
- 验证证据：当前加固版平均耗时 0.27328s，相对基线提速 8.55×。

5.4 多行函数签名进入语义模型

- 现象：参数列表跨行时，旧函数和方法模式可能漏收集声明。
- 根因：早期快速匹配路径限定参数位于单行。
- 解决方法：保留单行快速路径；检测到多行函数头时启用宽模式；函数收集、当前函数选择和声明类型检查使用统一分流条件。
- 验证证据：多行泛型函数、多行方法和多行调用进入固定回归及随机生成家族。

5.5 嵌套 *lambda* 参数可见性

- 现象：2-4 层嵌套 *lambda* 和高阶函数调用可能将合法参数识别为未定义名称。
- 根因：未闭合 *lambda* 的参数尚未进入普通函数作用域。
- 解决方法：扫描活动花括号栈，识别 { parameters => body }，将活动 *lambda* 参数加入表达式上下文，并传播期望函数类型。
- 验证证据：显式 *lambda*、推导 *lambda*、高阶函数、类静态方法和接口回调案例进入回归。

5.6 泛型接口传递继承与签名规范化

- 现象：接口继承链上的类型实参替换可能缺失；方法泛型参数名称差异可能形成虚假不匹配。
- 根因：接口要求需要沿继承链具体化，方法级类型参数还需要独立规范化。
- 解决方法：递归收集接口要求、沿链应用 substitution、使用 visited 控制循环、比较前执行 alpha-renaming。
- 验证证据：传递泛型接口继承、具体化实现和方法泛型签名进入随机回归。

5.7 重载方法引用歧义

- 现象：多个裸成员方法候选同时适用时，单个候选选择会掩盖歧义。
- 根因：方法引用缺少调用实参，普通调用匹配无法完成消歧。
- 解决方法：多个适用候选返回歧义；唯一候选构造函数类型；接收者泛型实参先完成替换。
- 验证证据：唯一方法引用和模糊方法引用分别进入生成器。

5.8 函数作用域与关键字边界

- 现象：函数参数可能进入后续函数上下文；foreign... 可能触发 for 语句路径。
- 根因：切换函数时状态清理范围不足，关键字检测只判断字符串前缀。

- 解决方法：切换函数时清空前一函数参数和不可变集合；StartsWithKeyword 同时检查关键字文本和后续标识符边界。
- 验证证据：跨函数作用域隔离和关键字前缀标识符纳入随机测试。

5.9 隐藏样例覆盖评估

- 现象：公开 50 例覆盖固定，复杂组合与输入分片仍需要独立验证。
- 解决方法：确定性随机生成合法程序和单点语义变异程序，使用完整程序 typechecker 标注，并执行 byte/random/line/cl100k/whole 五种分片。
- 验证证据：360 个程序、1800 次分片、0 失败；180 个合法程序实现零前缀误报。

6. 性能测试与 XGrammar 基线对比

6.1 实验环境与统计方法

项目	配置
镜像 ID	0522_cangjie_fragment_checker
Docker 镜像	docker.educg.net/compiler_system_challenge/cjchecker:20260522
操作系统	Ubuntu 22.04
服务器架构	ARM
样本	同一批 50 个公开错误案例
主指标	平台逐例耗时的算术平均值
提速计算	基线平均耗时 ÷ 对比版本平均耗时

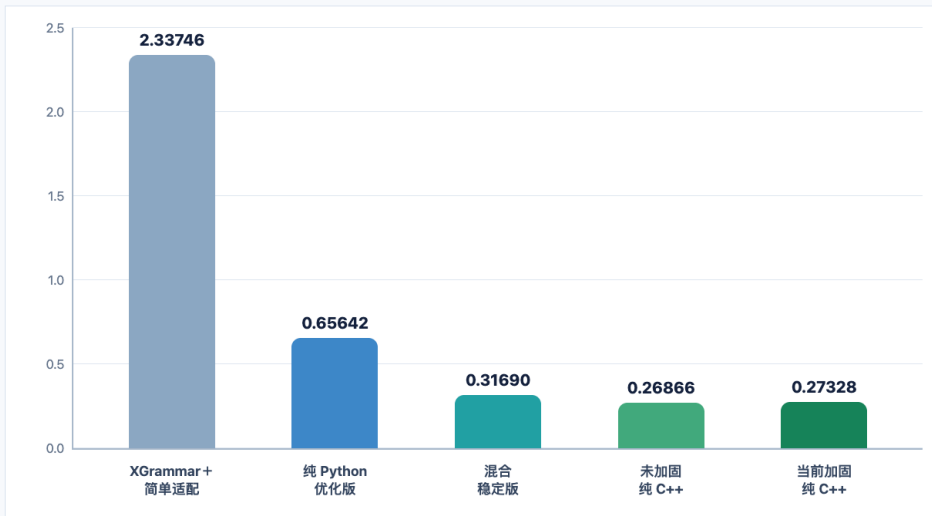
本文报告的功能和性能结果均来自上述赛题限定环境。平台平均耗时用于五阶段主对比；冷进程 p50/p95 用于三个内部原生版本辅助分析，两种统计口径分别呈现。

6.2 五阶段平台性能

性能演进 | 同一批 50 个公开案例的平台平均耗时

五阶段优化：最终加固版相对基线提速 8.55×

所有数据来自赛题限定环境；柱高为平台平均耗时，单位为秒。



8.55×

最终相对基线提速

平均耗时从 2.33746s 降至 0.27328s。

88.31%

平均耗时降低

50 个案例全部取得正向加速。

+1.72%

加固覆盖成本

相对未加固版，增加多行、lambda、接口、重载与作用域检查。

n=50 | 0522_cangjie_fragment_checker
Ubuntu 22.04 | ARM | AC

版本	样例数	累计耗时	平均耗时	相对基线
XGrammar + 简单适配	50	116.873s	2.33746s	1.00×
纯 Python 优化版	50	32.821s	0.65642s	3.56×
混合稳定版	50	15.845s	0.31690s	7.38×
未加固纯 C++ 版	50	13.433s	0.26866s	8.70×
当前加固纯 C++ 版	50	13.664s	0.27328s	8.55×

纯 Python 优化版中位数为 0.658s，p95 约 0.6962s，最小值 0.585s，最大值 0.708s；相对基线耗时降低 71.92%，50 个案例均取得加速。当前加固版最小值为 0.223s、最大值为 0.310s；相对基线平均耗时降低 88.31%，50 个案例全部取得加速。

上述逐例平台数据来自公开样例，检查器在公开标注的首错位置输出错误并终止，因此主要反映官方公开错误负载。合法长程序的完整流式行为另由项目语料、180 个随机合法程序、多分片测试和真实协议测试覆盖。混合稳定版与未加固纯 C++ 版只有已保存的 50 例平均值，累计耗时按 50 × 平均耗时 计算。

6.3 内部原生化版本的冷进程辅助数据

本队迭代版本	p50	p95	作用
混合稳定版	69.60ms	88.01ms	建立 C++ 协议、解码和语法入口，语义由 Python worker 执行
未加固纯 C++ 版	45.98ms	59.84ms	完成语义热路径 C++ 化
当前加固纯 C++ 版	53.49ms	71.34ms	增加多行、lambda、接口、重载和作用域检查

加固版以 1.72% 的平台平均耗时增幅换取更完整的隐藏样例式覆盖，最终仍保持相对基线 8.55× 的加速。

6.4 与类似方法和参考工作的对比

本节将“平台实测”和“文献能力分析”分开。平台速度倍数只用于实际运行过的五阶段版本；论文与外部项目采用定性能力对比，不推导跨硬件、跨任务的速度结论。

方案	主要目标	增量语法	类型/作用域	未完成前缀	泛型/lambda	运行方式	本文证据类型
竞赛配套标准 typechecker	完整程序解析与语义检查	否	是	需要完整结构，不能直接接受多数未闭合前缀	依竞赛子集实现	Lark 完整解析与检查	离线 oracle、早期完整重检路径
XGrammar v0.2.1 [4]	结构化生成中的形式语法约束	是	否	维护语法可行前缀	只覆盖其语法形式	GrammarCompiler / GrammarMatcher 状态推进	本项目运行时实测
每 token 完整重检基线	将完整 typechecker 用于前缀检查	弱，每轮重新开始	是	需要额外的补全/容错策略	取决于完整 typechecker	每轮扫描、Lark 解析和类型检查	本项目基线实测
Koo 等的自动机约束方法 [1]	处理 tokenizer 与形式语言约束的对齐	是	论文重点为形式语言约束	支持正则及确定性上下文无关语言前缀	未针对仓颉类型语义	自动机编译与状态推进	文献分析
DOMINO [2]	子词对齐、低侵入的约束生成	是	论文重点为形式语言约束	支持子词对齐的约束前缀	未针对仓颉类型语义	预计算、speculative decoding 与约束解码	文献分析
类型约束代码生成 [3]	在生成中利用类型系统降低编译错误	论文包含 prefix automata	是	在形式化语言和 TypeScript 扩展中处理类型可达性	能力随论文模型与 TypeScript 扩展而定	prefix automata、可居住类型搜索	文献分析
本作品	仓颉赛题子集的语法和语义前缀检查	是	是	稳定词法事件、partial probe 与不可恢复错误判定	支持赛题三档用例涉及形式	XGrammar + 本队缓存模型 / 活动语句 C++ 语义探测	平台、公开样例、随机与分片实测

标准 typechecker 对完整程序提供语义 oracle，面对未闭合表达式时需要额外的前缀容错；每轮重新解析会重复处理历史字符。XGrammar 高效维护语法前缀状态，其职责范围不包含仓颉类型、作用域和调用匹配。本作品复用 XGrammar 的通用语法自动机，在其上增加面向仓颉赛题子集的增量语义状态、PartialLexeme 保守处理和不可恢复错误判定。该组合同时保留语法约束、语义首错和合法未完成前缀三类需求。

7. 正确性与鲁棒性测试

7.1 验证方法

测试证据按 oracle 和可支持结论分层：组委会标注负责公开样例的精确首错；官方平台记录负责实际提交结果；完整程序 typechecker 负责随机程序最终合法性标签；mutation_start 负责约束非法变异不得在变化发生前被拒绝；相同字符流的五种分片负责验证最终结论一致性；sanitizer 负责已测输入中的内存安全与未定义行为。随机非法程序没有组委会提供的精确首错标签，因此本文不将其计入“精确首错”统计。

正确性证据 | 最终结论、拒绝时点与分片一致性

多维验证覆盖精确首错、合法前缀、随机变异与运行安全

所有结果来自项目记录的回归、随机生成器、真实协议和 sanitizer 验证。

精确首错	公开错误样例	50 / 50	检查首个不可恢复 token
固定语料	竞赛配套 + 项目语料	45 / 45 + 57 / 57	覆盖语义回归与项目维护案例
分片一致性	原生固定分片	66 / 66 × 4	逐字节、随机、cl100k、整段结果一致
随机稳健性	隐藏样例式生成	360 / 1800 / 0	程序数 / 分片执行数 / 失败数
合法前缀	180 个合法程序	0 误报	全部观测前缀保持可续写
运行安全	真实协议 + ASan/UBSan	48 / 48; 24 × 5	stdin/stdout 路径与内存安全检查
验收标准：错误首错准确，合法程序的每一个已观测前缀保持零误报。			

7.2 分层验收结果

测试类别	Oracle / 期望来源	验证内容	结果	可支持的结论
公开首错样例	组委会 wrong_error_positions 标注	精确首错 token	50 / 50	公开 50 个错误样例的首错轮次正确
平台测试	官方竞赛平台	实际构建、交互协议、平台 测试负载和超时	AC、25.00 分	当前加固版在 2026-08-05 02:59:28 的平台提交通过
合法随机程序	竞赛配套完整 typechecker	最终合法且全部观测前缀无 误报	180 程序 / 0 误 报	对生成样本保持保守接受
非法随机变异	生成器 + 完整 typechecker + mutation_start	最终拒绝，且不在单点变异 开始前拒绝	180 程序 / 0 失 败	对生成变异的最终结论正确，未 在未变异前缀提前误报
多分片一致性	同一 UTF-8 字符流	byte、random、line、 cl100k、whole 五种布局	360 × 5 = 1800 次 / 0 失败	最终合法性判断不依赖输入分片 方式
ASan/UBSan	编译器 sanitizer	内存安全和未定义行为	24 程序 × 5 分片 通过	已测输入中未发现相关运行时错 误

四个确定性随机种子 20260805、20260806、731991 和 991827 共生成 360 个程序，其中 180 个合法、180 个单点语义变异。每个程序使用五种分片布局执行，共 1800 次。所有程序的最终合法性与完整程序 oracle 一致；合法程序在全部观测前缀上未出现误报；非法程序未在 mutation_start 之前提前拒绝；不同分片布局的最终结论一致。覆盖家族包括多行泛型函数与调用、2-4 层嵌套 lambda、高阶泛型调用、重载与歧义方法引用、传递泛型接口继承、控制流、类/接口、注释、作用域隔离和关键字前缀标识符。

其他固定回归用于定位具体模块退化：

固定回归	结果	验证范围
竞赛配套语义语料	45 / 45	完整程序语义规则
项目语料	57 / 57	项目维护的功能组合
原生分片固定回归	66 / 66 × 4	byte、random、cl100k、whole 的固定用例一致性
context 变体	7 / 7	预加载符号、重载与成员格式
单元测试	34 / 34	核心局部规则
真实 solution 协议随机测试	48 / 48	stdin/stdout、token 解码与生产入口

7.3 代表性复现命令

```
./build.sh
python3 benchmark/differential_check.py --solution ./solution
python3 benchmark/native_fragment_differential.py
python3 benchmark/native_context_differential.py
python3 -m unittest discover -s tests
python3 benchmark/hidden_semantic_fuzz.py --seed 20260805 --cases-per-family 12
python3 benchmark/hidden_semantic_fuzz.py --seed 424242 --cases-per-family 4 --solution
./solution
python3 benchmark/hidden_semantic_fuzz.py --seed 868686 --cases-per-family 2 --sanitize
python3 benchmark/production_benchmark.py --solution ./solution
```

8. 为什么选择加固纯 C++

8.1 团队的工程取舍

未加固纯 C++ 版已经取得五阶段中的最低平台平均耗时 0.26866s。加固后，当前版本的平均耗时为 0.27328s，每例平均增加 0.00462s，50 例累计增加 0.231s，增幅约为 1.72%。本队接受这部分可控开销，原因在于赛题首先要求每一轮前缀判断保持正确；隐藏样例中的系统性误判会直接影响功能得分，也会削弱性能数据的实际意义。当前加固版仍比 XGrammar + 简单适配基线快 8.55×，因此这一取舍保留了接近一个数量级的总体性能优势。

我们的判断标准是：在性能优势已经达到稳定量级后，继续压缩几毫秒的收益低于补齐复杂语义组合的收益。加固工作把更多判断依据纳入持久状态和活动上下文，使引擎在合法前缀尚未闭合时保持可续写，在错误已经不可恢复时及时锁存失败。由此获得的正确性余量更适合面对公开样例之外的输入组合。

8.2 加固重点降低的潜在风险

加固主要面向隐藏样例中的组合性风险，覆盖四类问题：

- 跨行与未完成结构风险：多行函数签名、多行调用和嵌套表达式可能被物理换行提前截断。活动语句缓存按照括号、字符串、注释和提交边界维护完整语义片段，降低合法前缀被提前拒绝的概率。
- 高阶函数与 `lambda` 上下文风险：嵌套 `lambda` 参数在表达式闭合前尚未进入普通函数作用域。活动 `lambda` 扫描和期望函数类型传播为参数提供即时可见性，降低高阶调用中的未定义变量和错误类型推导。
- 泛型接口与重载消歧风险：接口类型实参需要沿继承链传递替换，方法级泛型参数需要规范化，裸成员方法引用还可能具有多个适用候选。递归要求收集、`alpha-renaming` 和歧义检测降低虚假签名不匹配及任意选取重载的风险。
- 作用域与词法边界风险：函数切换时残留参数可能泄漏到后续函数，关键字前缀标识符可能触发控制流规则。上下文清理和关键字边界检查降低跨函数污染与误识别。

这些问题具有共同特征：单项规则通常能够通过简单样例，跨越多行、`lambda`、泛型、接口、重载和作用域后更容易暴露状态不完整。加固版将相关组合纳入固定回归与确定性随机生成，最终完成 360 个程序、1800 次分片执行且失败数为 0，其中 180 个合法程序在全部观测前缀上保持零误报。该证据支持本队以 1.72% 平均耗时增幅换取更完整隐藏样例覆盖的选择。

9. 已知边界与适用范围

本作品面向赛题指定的仓颉子集和组委会提供的 context，文中结论不外推到完整仓颉 1.0.0 语言。

当前检查器采用工程化保守判定，不对全部可能后缀执行显式枚举或形式化可达性证明。公开精确首错、平台 AC 和项目测试用于验证赛题负载下的实际效果。

TokenEvent 当前负责稳定词法边界，语义检查主要结合完整可见源码、缓存声明模型、函数上下文和活动语句进行选择探测。完全由 TokenEvent 驱动的声明和表达式单次提交属于后续优化方向。

作用域能力以函数参数、函数体局部变量、函数间隔离、循环绑定和活动 lambda 上下文等已测试场景为主，文中结论不覆盖完整仓颉语言的全部嵌套块遮蔽、复杂闭包捕获和非常规生命周期组合。

结构提交点或上下文探测仍可能扫描较大范围源码，活动表达式 probe 也可能重新推导局部表达式。性能结论以公开 50 例在竞赛限定 Docker、Ubuntu 22.04、ARM 环境中的平台负载为准。

10. 第三方来源与原创贡献声明

10.1 可审计组件清单

版本列区分“固定版本”和 requirements.txt 中的“最低版本约束”。当前项目没有为范围依赖提供 lockfile，因此不把构建时实际解析到的版本伪写成固定版本。

组件	版本/约束	来源	许可证或使用依据	使用阶段	进入竞赛运行路径	是否包含其源码	最终提交框架中的形式	本队修改与适配
XGrammar	v0.2.1	官方 GitHub / PyPI	Apache-2.0	构建及运行	是	否	构建时安装，运行时动态链接	通用引擎未修改；本队编写仓颉 GBNF、C++ 入口和状态调用适配
Apache TVM FFI	apache-tvm-ffi>=0.1.9	Apache TVM FFI	Apache-2.0	构建及运行	是，作为 XGrammar 共享库依赖	否	XGrammar 的外部运行依赖	未修改；构建脚本完成定位、链接和 rpath 适配
tiktoken	tiktoken>=0.7.0	OpenAI tiktoken	MIT	构建期	否	否	构建时调用其公开 API 生成词表	未修改；本队工具读取 cl100k_base 编码并生成本地二进制查表
Lark	lark>=1.1.0	lark-parser/lark	MIT	开发期离线测试	否	否	仅用于开发测试，不进入生产框架	未修改；供竞赛配套 typechecker 解析完整程序
竞赛配套 typechecker	竞赛仓库 2026-06-07 公开快照	cangjie-fragment-checker	竞赛配套公开参考实现；上游快照未提供独立 LICENSE/NOTICE，本文不推测其通用开源许可证	开发期离线 oracle、差分、随机标注和实验复现	否	是	随最终代码框架提交的开发/测试材料，不进入生产运行路径	功能性适配涉及 5 个文件，原记录约新增 98 行、删除 76 行（不含统一来源注释）；整体均排除在本队原创成果和原创代码量之外，详见 10.3

requirements.txt 还保留 pydantic>=2.0、numpy>=1.24 和 typing-extensions>=4.9.0 作为 Python 测试/兼容依赖；当前 C++ 生产入口不导入这些包。首次在空环境构建时需要访问 Python 依赖源，build.sh 会检查并安装 tiktoken、XGrammar 和 TVM FFI。本文性能结果来自固定竞赛 Docker，因此不受后续 PyPI 版本变化影响；复现者应使用同一镜像或准备依赖缓存。

10.2 第三方能力、本队适配与独立实现边界

第三方原有能力

- XGrammar 的 GBNF 编译、GrammarCompiler 和 GrammarMatcher；
- TVM FFI 的原生共享库和 ABI 支持；
- tiktoken 的 cl100k_base 编码定义；
- Lark 的通用解析能力；
- 竞赛配套 typechecker 的完整程序解析与类型检查框架。

本队基于第三方完成的适配

- 面向赛题仓颉子集编写的 GBNF；

- XGrammar C++ 运行入口和逐字节状态推进；
- cl100k token ID 到 UTF-8 字节片段的本地二进制查表；
- context JSON 到原生二进制模型的转换；
- XGrammar/TVM FFI 的构建、动态链接和竞赛协议接入。

本队独立实现

- IncrementalLexer 以及 TokenEvent / PartialLexeme 分流；
- 原生 C++ 前缀语义状态与首次不可恢复错误锁存；
- 变量、作用域、控制流、表达式、调用、重载、泛型、lambda 和接口的赛题子集规则；
- ActiveStatementCache 的多行活动语句维护；
- 泛型接口传递替换、方法签名规范化和重载方法引用消歧；
- context/token 表生成工具，以及差分、分片、随机变异、协议和 sanitizer 测试工具。

以上“本队独立实现”和本文涉及的原创代码量，均明确排除 `third_party/cangjie_typechecker/` 中的上游代码及本地适配；该目录只作为已披露来源的开发/测试材料随代码框架提供。

10.3 竞赛配套 *typechecker* 的开发期用途与提交边界

`third_party/cangjie_typechecker` 基于组委会公开的 [cangjie-fragment-checker](#) 中的 `typechecker`。本项目记录所用版本为竞赛仓库 2026-06-07 公开快照；现有材料没有保留可进一步核验的 commit SHA，因此以公开链接、快照日期和下述修改文件清单共同标识。该副本用于完整程序解析、随机程序合法性标注、差分测试和实验复现，不参与 `solution` 的编译、链接或运行。

最终提交的代码框架包含 `third_party/cangjie_typechecker`，其定位是开发/测试材料，而不是生产运行依赖。当前生产运行路径由本队编写的 `cpp/solution.cpp`、`cpp/native_semantic.cpp`、面向赛题子集适配的仓颉 GBNF 和构建工具，以及外部安装的 XGrammar/TVM FFI 依赖构成。

本地副本相对组委会公开版本对 `builtin_context.py`、`checker.py`、`context.json`、`type_inference.py` 和 `type_services.py` 5 个文件作了功能性适配；扣除统一的来源注释后，原开发记录约为新增 98 行、删除 76 行。修改内容用于扩充迭代类型、字符串操作、默认构造器和接口实现等离线测试能力，不构成竞赛运行时前缀检查算法。此外，所有支持注释的 vendored Python/Lark 源文件均增加了不影响逻辑的统一来源头，因此按文件系统直接统计的修改文件数和新增行数会高于上述功能性修改统计。

该目录的上游代码及本地适配均不作为本队原创成果，也不计入本队原创代码量。可添加注释的源码文件均在头部标明来源；不支持注释的 `context.json` 由目录 README、本文及 `THIRD_PARTY_NOTICES.md` 统一归因。除该已披露参考目录外，团队源文件仅调用 XGrammar、TVM FFI、tiktoken 或 Lark 的公开 API，不包含从这些项目复制的实现代码。

10.4 最终提交框架清单

最终提交框架按以下边界确定，生产文件、开发期验证材料和外部依赖的归属分别列示：

路径或材料类别	进入最终提交框架	说明
cpp/	是	本队编写的生产 C++ 入口、增量词法和语义检查实现
grammar/	是	本队面向赛题子集编写的仓颉 GBNF；最初由 v0.53.x 文法整理，后按赛题子集和仓颉 1.0 规范校准，冲突时以赛题口径为准
tools/ 中的构建生成工具	是	本队工具，用于生成 context 和 token 二进制表
build.sh	是	本队编写的生产构建入口
context.json 及必要配置	是	构建原生预定义符号模型所需输入
README.md、设计文档和 THIRD_PARTY_NOTICES.md	是	使用、设计与第三方合规说明
third_party/cangjie_typechecker/	是（开发/测试材料）	基于竞赛配套公开参考实现并作少量适配，用于差分、随机标注和实验复现；不参与生产构建或运行，且不计入本队原创成果及原创代码量
本队编写的 benchmark/fuzz 工具	是（验证材料）	用于复现本文测试证据，不参与 solution 生产路径
旧 Python 运行路径	是（历史与调试材料）	用于差分和迭代说明，不属于当前加固纯 C++ 生产路径

11. 参考文献

1. Terry Koo, Frederick Liu, Luheng He. [Automata-based constraints for language model decoding](#). COLM 2024.
2. Luca Beurer-Kellner, Marc Fischer, Martin Vechev. [Guiding LLMs The Right Way: Fast, Non-Invasive Constrained Generation](#). ICML 2024, PMLR 235:3658–3673.
3. Niels Mündler, Jingxuan He, Hao Wang, Koushik Sen, Dawn Song, Martin Vechev. [Type-Constrained Code Generation with Language Models](#). PACMPL 9 (PLDI), 2025.
4. MLC-AI. [XGrammar v0.2.1](#), Apache License 2.0.
5. 仓颉编程语言社区. [仓颉编程语言 1.0.0 文档](#).
6. Apache Software Foundation. [Apache TVM FFI Documentation](#).
7. OpenAI. [tiktoken](#), MIT License.
8. 竞赛配套项目. [cangjie-fragment-checker](#).

12. 结论

本作品以 XGrammar 通用语法引擎和本队仓颉简单适配版本为起点，依次完成纯 Python 热路径优化、C++ 语法入口与 Python 语义混合、纯 C++ 语义迁移和隐藏样例式加固。当前加固版在竞赛限定 Ubuntu 22.04/ARM 环境中取得平台 AC，50 例平均耗时从基线的 2.33746s 降至 0.27328s，实现 8.55×、接近一个数量级的加速。

当前版本的技术贡献包括 TokenEvent 与 PartialLexeme 的增量词法分流、XGrammar 语法状态与基于缓存模型/活动语句的 C++ 语义探测协同、活动语句缓存、保守不可恢复错误判定、传递泛型接口替换和方法泛型签名规范化。通用编译技术为这些模块提供基础，本队工作聚焦其在 BPE/UTF-8/仓颉三重边界和逐 token 首错任务中的组合实现。公开精确首错、多语料、多分片、随机变异、合法前缀零误报、真实协议和 sanitizer 测试共同验证了本文所声明范围内的正确性与鲁棒性。

后续优化将聚焦持久化 ExprFrame / CallFrame、TokenEvent 驱动的声明单次提交、类型驻留和 scope-aware 名称索引，以继续降低结构提交点和局部表达式重建成本。

附录 A：逐例平台性能数据

下表单位均为秒。单例提速倍数按“XGrammar + 简单适配耗时 ÷ 当前加固纯 C++ 耗时”计算。

案例	XGrammar+ 简单适配	纯 Python 优化版	当前加固纯 C++	基线→加固提速
err_undefined	3.121	0.708	0.294	10.62×
err_assign_let	3.706	0.702	0.310	11.95×
err_arity	3.015	0.680	0.292	10.33×
err_if_not_bool	2.313	0.658	0.277	8.35×
err_break	2.891	0.669	0.289	10.00×
err_type_mismatch	2.604	0.669	0.287	9.07×
err_continue_outside_loop	2.604	0.670	0.293	8.89×
err_return_type_mismatch	0.862	0.585	0.223	3.87×
err_duplicate_var	0.916	0.596	0.224	4.09×
err_arraylist_toarray_assign	2.468	0.663	0.284	8.69×
err_unary_minus_non_numeric	2.196	0.648	0.277	7.93×
err_eq_incomparable	2.322	0.658	0.269	8.63×
err_rel_mixed_numeric	2.057	0.647	0.265	7.76×
err_mod_non_int64	2.008	0.645	0.263	7.63×
err_range_non_int	2.213	0.653	0.265	8.35×
err_for_not_iterable	2.133	0.646	0.262	8.14×
err_for_pattern_map_bad	2.333	0.664	0.274	8.51×
err_array_index_not_int64	2.991	0.689	0.288	10.39×

err_array_fill_type	2.753	0.676	0.283	9.73×
err_string_contains_arg	2.525	0.659	0.280	9.02×
err_ctor_call_mismatch	2.678	0.672	0.280	9.56×
err_generic_arity	2.476	0.666	0.282	8.78×
err_interface_not_implemented	0.925	0.593	0.227	4.07×
err_interface_sig_mismatch	0.942	0.593	0.223	4.22×
err_no_member	2.216	0.646	0.272	8.15×
err_unknown_named_arg	2.380	0.658	0.272	8.75×
err_index_non_array	1.981	0.645	0.262	7.56×
err_unary_not_non_bool	1.910	0.644	0.259	7.37×
err_while_not_bool	2.122	0.645	0.265	8.01×
err_arraylist_add_type	2.290	0.656	0.278	8.24×
err_hashmap_key_type	2.283	0.657	0.280	8.15×
err_bound_var_mismatch	2.062	0.644	0.266	7.75×
err_rel_unordered	2.091	0.647	0.265	7.89×
err_interface_as_value	1.829	0.648	0.264	6.93×
err_arith_non_numeric	2.618	0.665	0.281	9.32×
err_lambda_arg_arity_explicit	2.866	0.670	0.276	10.38×
err_lambda_return_type_explicit	2.465	0.660	0.273	9.03×
err_lambda_zero_body_explicit	2.006	0.643	0.265	7.57×
err_lambda_hof_explicit	3.725	0.694	0.295	12.63×
err_lambda_interface_callback_explicit	2.919	0.698	0.302	9.67×
err_lambda_in_class_static_explicit	2.087	0.657	0.274	7.62×
err_lambda_param_narrow_explicit	2.419	0.655	0.275	8.80×
err_lambda_infer_ambiguous_1	2.378	0.657	0.270	8.81×
err_lambda_infer_ambiguous_2	2.991	0.668	0.286	10.46×
err_lambda_infer_wrong_return_1	1.996	0.636	0.265	7.53×
err_lambda_infer_wrong_return_2	3.093	0.669	0.282	10.97×
err_lambda_infer_collection_1	2.201	0.658	0.277	7.95×
err_lambda_infer_collection_2	2.384	0.664	0.278	8.58×
err_lambda_infer_class_helper	2.139	0.657	0.278	7.69×
err_lambda_infer_interface_helper	2.370	0.671	0.293	8.09×

附录汇总

指标	XGrammar+简单适配	纯 Python 优化版	当前加固纯 C++
样例数	50	50	50
累计耗时	116.873s	32.821s	13.664s
平均耗时	2.33746s	0.65642s	0.27328s
中位数	2.3275s	0.658s	0.2765s
p95（线性插值）	3.1084s	0.6962s	0.29455s
最小值	0.862s	0.585s	0.223s
最大值	3.725s	0.708s	0.310s